

โครงงาน
Cha Cha Mala

จัดทำโดย

นางสาวพิมพ์ลภัส คงสุนทร 6704062617172

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของโปรเจกต์

ในปัจจุบันอาหารประเภทหม่าล่าต้มสายพานและหม้อไฟได้รับความนิยมอย่างมากในสังคมไทยและระดับสากล ด้วยรสชาติที่เป็นเอกลักษณ์และความหลากหลายของวัตถุดิบและน้ำซุป ผู้พัฒนาจึงได้เกิดแนวคิดในการนำบรรยากาศการบริการและปรุงอาหารหน้าร้านหม่าล่า มาสร้างเป็นเกมคอมพิวเตอร์แนวจำลองการจัดการร้านอาหาร (Time Management / Cooking Simulation) โดยพัฒนาด้วยภาษา Java และเทคโนโลยี Java Swing ซึ่งช่วยให้ผู้พัฒนาได้บูรณาการความรู้ด้านการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP), สถาปัตยกรรมการแยกข้อมูลและการแสดงผล (Model-View Separation), และการพัฒนา Graphical User Interface (GUI) แบบ Custom Rendering เข้าด้วยกันอย่างเป็นรูปธรรม

1.2 ประเภทของโครงการ

เกมจำลองการบริหารร้านอาหาร (Simulation / Time Management Game) แบบ 2 มิติ พัฒนาเป็น Java Desktop Application ด้วย Swing ควบคุมด้วยเมาส์

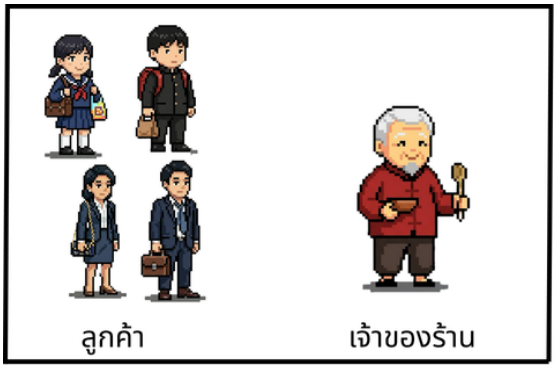
1.3 ประโยชน์

1. พัฒนาทักษะการคิดและการแก้ปัญหา
2. ฝึกสมาธิให้จดจ่อกับสิ่งที่ทำ
3. เสริมสร้างทักษะการบริหารเวลา
4. คลายเครียดและสร้างความเพลิดเพลิน
5. ผู้พัฒนาได้ฝึกประยุกต์ใช้หลักการ OOP และการพัฒนา GUI ด้วย Java

1.4 ขอบเขตของโครงการ (Project Scope / Proposal)

1. ระบบการแสดงผล (Display & Graphics):
    - แสดงผลผ่านหน้าต่างเดสก์ท็อปขนาด 820 x 620 พิกเซล แบบคงที่ (Non-resizable)
    - รองรับ Custom Pixel Font (TA GBA monothai_MAX.ttf) สำหรับแสดงผลภาษาไทยและอังกฤษอย่างถูกต้อง
    - แสดงผลรูปภาพ Custom Sprites สไตล์พิกเซล เช่น หน้าร้าน, ลูกค้า 4 กลุ่ม, เตาแก๊ส, หม้อซุป 3 สี, วัตถุดิบ 6 ชนิด, เครื่องคิดเงิน, และหลอดความอดทน
2. ระบบสถานะเกม (Game States):
    - มีสถานะเกม 6 สถานะหลัก: MENU, SETTINGS, DIALOGUE, DAY_GOAL_BRIEF, PLAYING, และ SUMMARY
3. ระบบการเล่น (Gameplay System):
    - ระบบลูกค้าสุ่มสลับเพศและวัย 4 รูปแบบ (นักเรียนชาย/หญิง, วัยทำงานชาย/หญิง)
    - บับเบิ้ลออเดอร์แสดงรายการวัตถุดิบที่ต้องการ (สุ่ม 1 ถึง 3 ชนิดตามวันที่) และซุป 1 ใน 3 แบบ (หม่าล่า, กระดูกหมู, มะเขือเทศ)
    - ระบบทำอาหาร: เลือกหม้อน้ำซุปตั้งบนเตาแก๊ส -> คลิกใส่วัตถุดิบลงหม้อ -> ปรุงสุก -> เสิร์ฟและรับเงินที่เครื่องคิดเงิน
    - ระบบจับเวลานับถอยหลังความอดทนของลูกค้า (8 วินาที/คน)
4. ระบบเป้าหมายและเงื่อนไข (Winning / Losing Conditions):
    - เป้าหมายยอดเงินในแต่ละวันคำนวณตามสูตร $DayTarget = 500 + (Day \times 250)$
    - เสิร์ฟถูกต้องรับเงินตามจำนวนถาด ($125 \times N$), เสิร์ฟผิดปรับเงิน 300 บาท, ลูกค้ารอจนหมดเวลาปรับเงิน 200 บาท
    - เงื่อนไขแพ้: หากเงินคงเหลือในร้านติดลบถึง -$1,000 จะถูกปรับสถานะเป็นร้านล้มละลาย (Game Over) และต้องเริ่มใหม่จาก Day 1

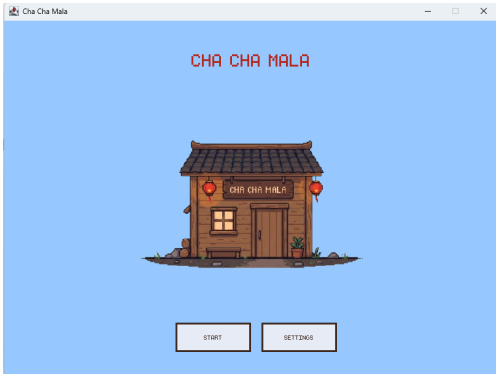
ตัวละครในเกม



หน้าเริ่มเล่น Day 1



หน้า Menu เกม



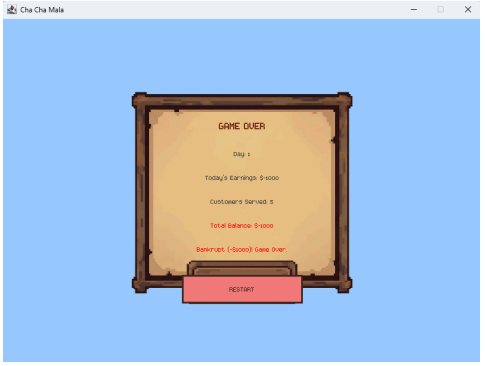
หน้าเริ่มเล่นเกม



หน้าสอนเล่น / อธิบายเกม



หน้า Game Over



หน้าทำภารกิจสำเร็จ เริ่มวันถัดไป



ตารางแผนการทำงานเดือนกันยายน-ตุลาคม

ลำดับ	รายการ	1-5 ก.ย.	6-15 ก.ย.	16พ.ย.-25ต.ค.	26ต.ค.-31ต.ค.
1	หารูปตัวละครและจัดทำกราฟิก				
2	ศึกษาเอกสารและข้อมูล				
3	เขียนโปรแกรม				
4	จัดทำเอกสาร				
5	ตรวจสอบและแก้ไขโครงงาน				

บทที่ 2 ส่วนการพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

เนื้อเรื่อง: เจ้าของร้านชาชาหม่าล่าปวดหลัง จึงมอบร้านให้ผู้เล่นดูแลต่อ ผู้เล่นต้องเปิดร้านวันแรกและทำยอดขายให้ถึงเป้าหมายในแต่ละวัน

วิธีการเล่น

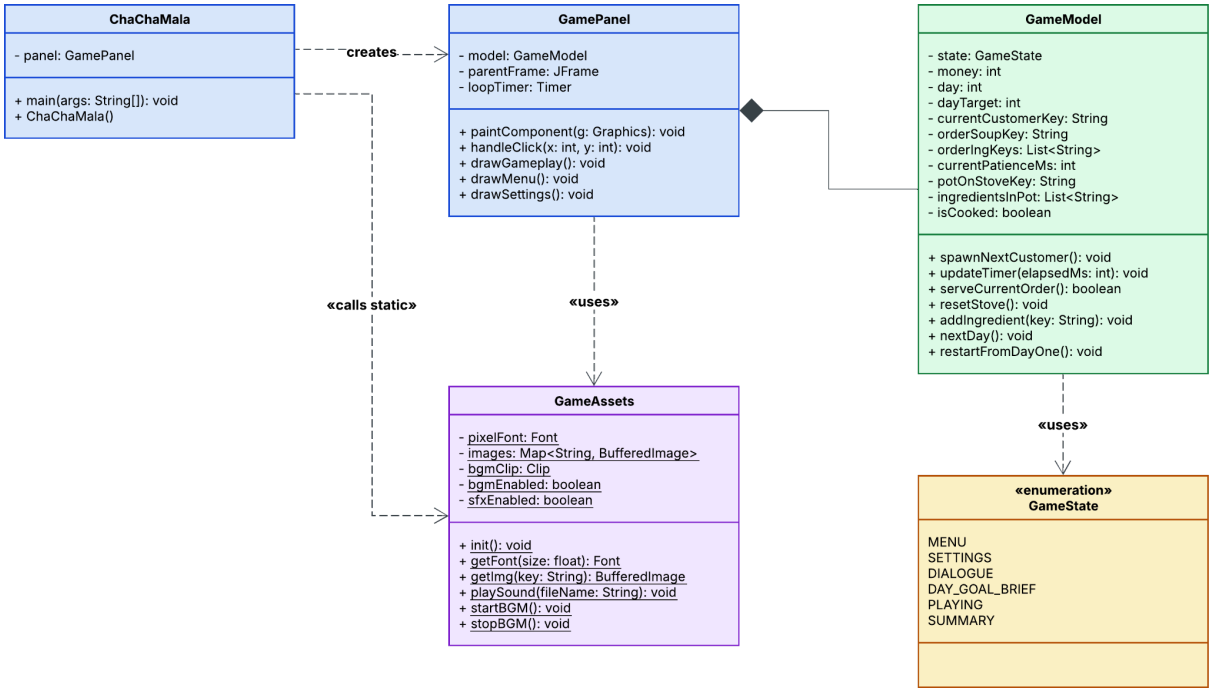
1. กด START แล้วอ่านบทสนทนาของเจ้าของร้านจนจบ จากนั้นกด NEXT เพื่อดูเป้าหมายประจำวัน และกด START COOKING!
2. ดูใบออเดอร์ในกรอบคำพูดเหนือหัวลูกค้า (ชนิดซุป และวัตถุดิบที่ต้องการ)
3. คลิกหม้อซุปให้ตรงชนิดเพื่อวางบนเตา
4. คลิกวัตถุดิบที่ลูกค้าสั่งใส่ลงหม้อ (ต้องวางหม้อก่อน)
5. คลิกเตาแก๊สเพื่อต้มให้สุก
6. คลิกเครื่องคิดเงินเพื่อเสิร์ฟและรับเงิน
7. ดูแถบเวลาของลูกค้า ถ้าแถบหมดก่อนเสิร์ฟ จะถูกหักเงิน

กติกาคะแนน

เหตุการณ์	ผลลัพธ์
เสิร์ฟอาหารถูกต้อง	ได้เงิน = จำนวนวัตถุดิบในออเดอร์ * 125
เสิร์ฟอาหารผิด	หักเงิน 300
ลูกค้ารอจนหมดเวลา (8 วินาที)	หัก 200
ยอดเงินถึงเป้าหมาย (500 + วัน * 250)	ผ่านวัน ไปวันถัดไป
เงินสะสมถึง -1000	Game Over เริ่มใหม่วันที่ 1

2.2 คลาสไดอะแกรม (Class Diagram)

แบ่งโครงสร้างคลาสออกเป็นโมดูลที่มีหน้าที่รับผิดชอบเฉพาะเจาะจง ตามแนวคิดสถาปัตยกรรมแบบ MVC (Model-View-Controller) ซึ่งประกอบด้วยคลาสหลัก 5 ส่วนที่ทำงานประสานกันดังนี้



1. คลาส ChaChaMala (Main Window Container)
ทำหน้าที่เป็นหน้าต่างหลักของแอปพลิเคชัน (Top-level Window) และเป็นจุดเริ่มต้นการรันโปรแกรม

- การสืบทอดคุณสมบัติ (Inheritance): สืบทอดมาจากคลาส javax.swing.JFrame
- หน้าที่และความรับผิดชอบ:
    - กำหนดการตั้งค่าสภาพแวดล้อมของหน้าต่างโปรแกรม เช่น ชื่อไตเติลบาร์ ("Cha Cha Mala"), กำหนดขนาดคงที่ 820×620 พิกเซล, ปิดการปรับขนาดหน้าต่าง (setResizable(false)), จัดตำแหน่งกึ่งกลางหน้าจอ (setLocationRelativeTo(null)), และกำหนดการปิดโปรแกรม (setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE))
    - สั่งเริ่มต้นโหลดทรัพยากรภาพและฟอนต์ผ่านคำสั่ง GameAssets.init() รวมถึงเริ่มเล่นเพลงประกอบวนลูปด้วย GameAssets.startBGM()

    - สร้างอินสแตนซ์ของพาเนลแสดงผล GamePanel และติดตั้งลงในตัวเฟรม
    - มีเมธอด main(String[] args) ทำหน้าที่รันโปรแกรมบน Event Dispatch Thread (EDT) ผ่าน SwingUtilities.invokeLater() เพื่อความปลอดภัยของการจัดการเธรดในระบบ Swing

2. คลาส GamePanel (View & Controller)
ทำหน้าที่เป็นทั้งผืนผ้าใบในการวาดองค์ประกอบกราฟิกทั้งหมดบนหน้าจอ (View) และเป็นตัวรับสัญญาณอีเวนต์การคลิกเมาส์จากผู้เล่น (Controller)

- การสืบทอดคุณสมบัติ (Inheritance): สืบทอดมาจากคลาส javax.swing.JPanel
- แอตทริบิวต์หลัก (Attributes):
    - model : GameModel — อ็อบเจกต์เก็บข้อมูลสถานะ ตรรกะ และเงื่อนไขการเล่นทั้งหมดของเกม
    - parentFrame : JFrame — ตัวแปรอ้างอิงถึงหน้าต่างหลัก
    - loopTimer : javax.swing.Timer — ตัวนับเวลา Game Loop ทำงานทุก 50 มิลลิวินาที (20 รอบต่อวินาที) เพื่อหักลบเวลาความอดทนของลูกค้า และสั่งรีเฟรชหน้าจอ (repaint()) แบบเรียลไทม์
- เมธอดสำคัญ (Methods):
    - GamePanel(JFrame parentFrame) (Constructor): กำหนดขนาด PreferredSize ผูกตัวดักฟังการคลิกเมาส์ (MouseAdapter) และเริ่มการทำงานของ loopTimer
    - paintComponent(Graphics g): Override เมธอดเดิมของ JPanel เพื่อควบคุมการวาดกราฟิกทั้งหมดตามสถานะของเกม (GameState) ผ่าน Graphics2D
    - กลุ่มเมธอดเรนเดอร์หน้าจอ (Screen Renderers):
        - drawSkyBackground(): วาดภาพพื้นหลังท้องฟ้า
        - drawMenu(): วาดหน้าเมนูหลัก ชื่อร้าน อาคารร้าน และปุ่ม START / SETTINGS
        - drawSettings(): วาดหน้าต่างตั้งค่าเปิด/ปิดเสียงเพลง (BGM) และเสียงเอฟเฟกต์ (SFX)
        - drawDialogue(): วาดบทสนทนาแนะนำตัวของลุงเจ้าของร้าน

        - drawDayBrief(): วาดป๊อปอัปแจ้งเป้าหมายยอดขายประจำวันและเวลาความอดทน
        - drawGameplay(): วาดหน้าโต๊ะเคาน์เตอร์, ตัวละครลูกค้า, บับเบิ้ลออเดอร์, ถาดวัตถุดิบ 6 ถาด, เตาแก๊สและหม้อต้ม, หม้อซุป 3 สี, เครื่องคิดเงิน, หลอดเวลา และยอดเงินสะสม
        - drawSummary(): วาดกระดานสรุปผลรายได้ จำนวนลูกค้า และสถานะผ่านวันหรือล้มละลาย (Game Over)
        - drawPixelBtn(), drawWrappedText(): เมธอดเสริมสำหรับวาดปุ่มกดสไตล์พิกเซลและตัดคำข้อความขึ้นบรรทัดใหม่อัตโนมัติ
    - handleClick(int x, int y): ตรวจสอบพิกัดการคลิกของผู้ใช้เทียบกับขอบเขต (Bounding Box) ของปุ่ม ถาดวัตถุดิบ หม้อซุป เตาแก๊ส และเครื่องคิดเงิน แล้วสั่งให้อัปเดตสถานะใน GameModel

3. คลาส GameModel (Business Logic & Data Model)
ทำหน้าที่เก็บข้อมูลสถานะ ตัวแปรระบบ สถิติการเงิน กฎเกณฑ์ และตรรกะทางธุรกิจของเกมทั้งหมด โดยไม่ขึ้นกับระบบแสดงผล

- แอตทริบิวต์หลัก (Attributes):
    - ข้อมูลสถานะและระบบเงิน: state : GameState, money : int, day : int, dayTarget : int, dayEarnings : int, customersServedToday : int
    - ข้อมูลลูกค้าและออเดอร์: currentCustomerKey : String, orderSoupKey : String, orderIngKeys : List<String>, currentPatienceMs : int (เวลาความอดทน 8,000 มิลลิวินาที)
    - ข้อมูลการปรุงบนเตา: potOnStoveKey : String, ingredientsInPot : List<String>, isCooked : boolean
    - บทสนทนา: dialogueTexts : String[], dialogueIndex : int
- เมธอดสำคัญ (Methods):
    - spawnNextCustomer(): สุ่มเลือกลูกค้า ชนิดน้ำซุป และวัตถุดิบตามระดับความยากของวัน (Day 1 สั่ง 1 ถาด, Day 2 สั่ง 1-2 ถาด, Day 3 ขึ้นไปสั่ง 1-3 ถาด) พร้อมรีเซ็ตเวลาความอดทน

- updateTimer(int elapsedMs): หักลบเวลาความอดทนของลูกค้า หากเวลาหมดลงเหลือ 0 จะหักเงิน $200 เล่นเสียงผิดพลาด และสลับลูกค้าคนถัดไป
- serveCurrentOrder() : boolean: ตรวจสอบความถูกต้องของน้ำซุปและวัตถุดิบในหม้อเทียบกับออเดอร์ หากถูกต้องจะคำนวณเงินรางวัลตามจำนวนถาด ($125 \times N$) หากผิดจะถูกปรับเงิน $300
- resetStove(), addIngredient(String key): จัดการสถานะการปรุงอาหารบนเตาแก๊ส
- nextDay(), restartFromDayOne(), prepareDayBrief(): ควบคุมการเปลี่ยนผ่านของวันและรีเซ็ตค่าเริ่มต้น
- กลุ่มเมธอด Getter และ Setter สำหรับเข้าถึงตัวแปรตามหลัก Encapsulation

4. คลาส GameAssets (Static Resource Manager)
ทำหน้าที่เป็นตัวจัดการทรัพยากรภาพ ฟอนต์ และระบบเสียงทั้งหมดแบบศูนย์กลาง โดยเป็นคลาส Utility ที่รวบรวมฟังก์ชันและตัวแปรแบบ static (ตามมาตรฐาน UML จะต้องขีดเส้นใต้ที่ชื่อสมาชิก)

- แอตทริบิวต์สถิต (Static Attributes):
  - pixelFont : Font — ฟอนต์สไตล์พิกเซล (TA GBA monothai_MAX.ttf) พร้อมระบบ Fallback เป็น Monospaced
  - images : Map<String, BufferedImage> — แคชจัดเก็บภาพ Sprite ทั้งหมดของเกม (พื้นหลัง, ตัวละคร, วัตถุดิบ, อุปกรณ์)
  - bgmClip : Clip — คลิปเสียงดนตรีพื้นหลัง
  - bgmEnabled : boolean, sfxEnabled : boolean — สถานะเปิด/ปิดเสียง BGM และ SFX
- เมธอดสถิต (Static Methods):
  - init(): โหลดฟอนต์และรูปภาพทั้งหมดจากพาธ images/ หรือ assets/ เก็บลงในหน่วยความจำ
  - getFont(float size) : Font: คืนค่าฟอนต์ตามขนาดที่ระบุ
  - getImg(String key) : BufferedImage: ดึงรูปภาพตามคีย์ชื่อภาพ
  - playSound(String fileName): เล่นเสียงประกอบ (SFX) ผ่านมัลติเธรดแยกเพื่อป้องกันไม่ให้หน้าจอ GUI กระตุก

- startBGM(), stopBGM(): เริ่มเล่นหรือหยุดดนตรีพื้นหลังแบบวนซ้ำต่อเนื่อง (Clip.LOOP_CONTINUOUSLY)

5. โครงสร้าง Enum GameState (State Machine)
ใช้สำหรับระบุและควบคุมสถานะการทำงานของหน้าจอเกม ประกอบด้วย 6 สถานะหลัก:

1. MENU — หน้าเมนูหลักของเกม
2. SETTINGS — หน้าจอตั้งค่าเปิด/ปิดเสียง
3. DIALOGUE — หน้าจอบทสนทนาแนะนำตัวของลุงเจ้าของร้าน
4. DAY_GOAL_BRIEF — หน้าต่างแจ้งเป้าหมายยอดขายประจำวัน
5. PLAYING — หน้าจอจำลองการปรุงอาหารและการขายจริง
6. SUMMARY — หน้าต่างสรุปผลยอดเงินประจำวัน หรือแจ้งเตือน Game Over หากล้มละลาย

ความสัมพันธ์ระหว่างคลาส (Class Relationships Summary)

1. Dependency: คลาส ChaChaMala เป็นผู้สร้าง (Instantiate) อ็อบเจกต์ GamePanel เพื่อนำไปบรรจุลงในหน้าต่างหลัก
2. Composition : คลาส GamePanel ถือครองอ็อบเจกต์ GameModel แบบ final (รูปข้าวหลามตัดทึบสีดำแนบติดกับขอบกล่อง GamePanel ซึ่งเป็นฝ่าย Owner) แสดงว่าวงจรชีวิตของ Model ขึ้นตรงกับ Panel
3. Usage / Static Dependency : คลาสหน้าจอเรียกใช้งานตัวแปรและเมธอดสถิตใน GameAssets โดยตรงเพื่อดึงภาพ ฟอนต์ และสั่งเล่นเสียง
4. Usage : คลาส GameModel อ้างอิงและใช้ค่าคงที่ใน Enum GameState ในการตรวจสอบและควบคุมสถานะการทำงานของเกม

2.3 รูปแบบการพัฒนา Application / Applet

โครงการนี้พัฒนาในรูปแบบ Java Desktop Application (Swing Desktop Application) ทำงานผ่านเมธอด main() และใช้ JFrame เป็นคอนเทนเนอร์หลัก ไม่ใช่ Applet (เนื่องจากเทคโนโลยี Applet ถูกตัดออกจากมาตรฐาน Java ยุคใหม่) มีการจัดโครงสร้างแบบจำลองสถาปัตยกรรม MVC (Model-View-Controller) ย่อยภายในตัว:

- Model: รับผิดชอบโดยคลาส GameModel เก็บ Data และ Rule ของเกม
- View: รับผิดชอบโดยเมธอด paintComponent() และเมธอด draw...() ใน GamePanel
- Controller: รับผิดชอบโดย MouseAdapter ในการดักจับพิกัดคลิกและส่งคำสั่งไปอัปเดตโมเดล

2.4 อธิบายส่วนของโปรแกรมตามหลัก OOP

1) Constructor

Constructor คือเมธอดพิเศษที่มีชื่อเดียวกับคลาส ใช้ในการกำหนดค่าเริ่มต้นให้กับอ็อบเจกต์เมื่อถูกสร้างขึ้นด้วยคำสั่ง new

```
public ChaChaMala() {
    GameAssets.init();
    GameAssets.startBGM();

    setTitle("Cha Cha Mala");
    setSize(820, 620);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setResizable(false);

    GamePanel panel = new GamePanel(this);
    add(panel);
}
```

รวมถึง Constructor ของ GamePanel(JFrame parentFrame) ที่ตั้งค่าขนาด Dimension เปิดการโฟกัส และผูก Mouse Listener กับ Swing Timer

2) Encapsulation (การห่อหุ้มและการซ่อนข้อมูล)

Encapsulation คือการปกป้องข้อมูลสมาชิก (Attributes) ภายในคลาสไม่ให้ถูกเข้าถึงหรือเปลี่ยนแปลงโดยตรงจากภายนอก โดยประกาศตัวแปรเป็น private และเปิดให้เข้าถึงผ่านเมธอด Getter / Setter

```
private GameState state = GameState.MENU;
private int money = 0;
private int day = 1;
private String potOnStoveKey = null;

public int getMoney() { return money; }
public GameState getState() { return state; }
public void setState(GameState state) { this.state = state; }
```

3) Composition (ความสัมพันธ์แบบประกอบกัน "Has-A")

Composition คือความสัมพันธ์ระหว่างคลาสที่อ็อบเจกต์หนึ่งถูกสร้างและเป็นส่วนประกอบภายในอีกอ็อบเจกต์หนึ่ง หากอ็อบเจกต์หลักถูกทำลาย อ็อบเจกต์ย่อยจะหมดหน้าที่ไปด้วย

```
class GamePanel extends JPanel {
    private final GameModel model = new GameModel();
    private final Timer loopTimer;
    ...
}
```

4) Polymorphism (การพ้องรูป)

Polymorphism คือความสามารถในการแสดงพฤติกรรมที่แตกต่างกันผ่านอินเทอร์เฟซหรือเมธอดชื่อเดียวกัน

```
SwingUtilities.invokeLater(new Runnable() {
    @Override
    public void run() {
        new ChaChaMala().setVisible(true);
    }
});
```

5) Inheritance (การสืบทอดคุณสมบัติ)

Inheritance คือการที่คลาสลูกสืบทอดตัวแปรและเมธอดมาจากคลาสแม่ เพื่อนำมาต่อยอดการทำงานโดยไม่ต้องเขียนขึ้นใหม่ทั้งหมด

```
public class ChaChaMala extends JFrame {
```

```
public class GamePanel extends JPanel {
    private final GameModel model;
    private final Timer loopTimer;
```

2.5 หน้าจอ GUI, ส่วนประกอบ และโครงสร้าง Component
โครงสร้างหน้าจอ GUI (Screen Views) ที่ถูกวาดผ่าน Graphics2D:

- Main Menu: แสดงป้ายชื่อร้านอักษรพิกเซล, ภาพหน้าร้าน, ปุ่มสี่เหลี่ยมพิกเซล START และ SETTINGS
- Settings: กล่องข้อความแสดงตัวเลือกเปิด/ปิดเสียงดนตรี (BGM) และเสียงประกอบ (SFX) พร้อมปุ่ม BACK
- Dialogue: กล่องคำพูดของลุงเจ้าของร้านด้านล่าง พร้อมรูปเจ้าของร้านและข้อความแนะนำการเล่น
- Day Goal Brief: ป๊อปอัปแจ้งเป้าหมายยอดขายประจำวัน ข้อมูลความอดทนของลูกค้า และปุ่ม START COOKING!
- Gameplay:
    - Header/HUD: กล่องเหรียญแสดงยอดเงินสะสมมุมซ้ายบน, ข้อความ Goal ประจำวัน, และปุ่ม MENU
    - Counter Table & Background: โต๊ะเคาน์เตอร์แบ่งครึ่งบน/ล่าง โดยให้ตัวละครลูกค้ายืนอยู่ด้านหลังเคาน์เตอร์
    - Interactive Trays: ถาดวัตถุดิบ 6 ถาดทางซ้ายมือ (เนื้อ, หมู, ลูกชิ้น, เห็ด, ผัก, เต้าหู้)
    - Stove Area: เตาแก๊สพกพาตรงกลาง รองรับการวางหม้อซุปและแสดงวัตถุดิบข้างใน
    - Soup Cauldrons: หม้อซุป 3 สีทางขวา (หม่าล่า, กระดูกหมู, มะเขือเทศ)

    - Cash Register: เครื่องคิดเงินแคชเชียร์ทางขวาสุด สำหรับกดเสิร์ฟและชำระเงิน
    - Customer Bubble & Timer: บับเบิ้ลออเดอร์ลอยเหนือตัวละคร แสดงรูปวัตถุดิบ $x1$ พร้อมหม้อซุปที่ต้องการ และหลอดเวลาสีเขียว/แดง
- Summary/Game Over: หน้าต่างกระดานไม้สรุปรายได้ประจำวัน จำนวนลูกค้า ยอดเงินรวม และปุ่ม NEXT DAY หรือ RESTART

2.6 อธิบาย Event Handling ที่มีในโปรแกรม

โปรแกรมมีการจัดการ Event 2 รูปแบบหลัก:

- Mouse Event Handling (MouseListener / MouseAdapter):
    - ใช้การดักจับอีเวนต์การคลิกเมาส์ผ่าน mouseClicked(MouseEvent e) ใน GamePanel
    - ตรวจจับตำแหน่งแกน X และ Y ของเคอร์เซอร์เมาส์เปรียบเทียบกับขอบเขตพิกัด (Bounding Box / Hitbox) ของแต่ละสถานะเกม เช่น
        - การคลิกปุ่มสี่เหลี่ยม: ตรวจสอบแบบ Axis-Aligned Bounding Box (AABB) เช่น x >= btnX && x <= btnX + btnW && y >= btnY && y <= btnY + btnH
        - การคลิกหม้อซุปทรงกลม: ใช้สูตรหาระยะทางแบบ Euclidean Distance ด้วย Math.hypot(x - cx, y - cy) <= radius
    - เมื่อคลิกถูกตำแหน่ง ระบบจะเปลี่ยนสถานะเกม, เล่นเสียงเหรียญ (coin.wav), หรือส่งคำสั่งไปยัง Model เช่น model.addIngredient() หรือ model.serveCurrentOrder()
- Timer Event Handling (Action Event via javax.swing.Timer):
    - ใช้ loopTimer = new Timer(50, e -> { ... }) ยิงสัญญาณ Event ทุกๆ 50 มิลลิวินาที (20 Tick ต่อวินาที)
    - ทำหน้าที่เป็น Game Loop คอยสั่งให้ model.updateTimer(50) หักลบค่าความอดทนของลูกค้า และเรียก repaint() เพื่อรีเฟรชหน้าจอกราฟิกอย่างต่อเนื่อง

2.7 อธิบายอัลกอริทึมที่สำคัญในโปรแกรม

1) Customer Spawning & Multi-Tray Generation Algorithm
อัลกอริทึมสุ่มออเดอร์ของลูกค้าตามระดับความยากของวัน (Day Progression):

- สุ่มเลือกลูกค้า 1 จาก 4 แบบ และน้ำซุป 1 จาก 3 แบบ
- กำหนดจำนวนถาดวัตถุดิบ (maxTrays): Day 1 สั่ง 1 ถาด, Day 2 สั่ง 1-2 ถาด, Day 3 ขึ้นไปสั่ง 1-3 ถาด
- สุ่มเลือกวัตถุดิบจากลิสต์โดยใช้ Loop ตรวจสอบไม่ให้วัตถุดิบในบับเบิ้ลซ้ำกัน (!orderIngKeys.contains(selectedIng))

2) Order Verification & Dynamic Pricing Algorithm
อัลกอริทึมการตรวจสอบความถูกต้องของอาหารที่ปรุง:

- ตรวจสอบว่าน้ำซุปบนเตาตรงกับออเดอร์หรือไม่ (potOnStoveKey.equals(orderSoupKey))
- วนลูปตรวจสอบว่าวัตถุดิบที่ลูกค้าสั่งทั้งหมดมีอยู่ในหม้อต้มครบถ้วนหรือไม่
- คำนวณรายได้แบบไดนามิก: หากถูกต้องจะได้รับเงินตามจำนวนถาด ($Earn = 125 \times N$) หากผิดจะถูกปรับเงิน $300

3) Dynamic Patience Timer Algorithm
อัลกอริทึมจับเวลาความอดทนของลูกค้า:

- กำหนดค่าเริ่มต้นไว้ที่ 8,000 มิลลิวินาที เมื่อเกมอยู่ในสถานะ PLAYING ระบบจะหักลบเวลาทีละ 50 ms
- คำนวณอัตราส่วน Ratio เพื่อปรับขนาดและสีของหลอดเวลา (หาก Ratio > 0.4 เป็นสีเขียว, น้อยกว่านั้นเปลี่ยนเป็นสีแดง)
- หากเวลาเหลือ 0 ms จะหักเงิน $200 เล่นเสียงผิดพลาด และสลับเป็นลูกค้าคนถัดไป

บทที่ 3 สรุป

3.1 ปัญหาที่พบระหว่างการพัฒนา

1. ปัญหาการจัดการพิกัดภาพกราฟิก (Layering & Coordinate Alignment): การวาดองค์ประกอบหลายชิ้นด้วย Graphics2D ประสบปัญหาการทับซ้อน เช่น ตัวละครลูกค้าลอยอยู่เหนือโต๊ะเคาน์เตอร์ แก้ไขโดยการแยกวาดภาพพื้นหลังเฉพาะส่วนโต๊ะเคาน์เตอร์ไม้มาทับท่อนล่างของตัวละคร (Counter Depth Clipping)
2. ปัญหาขนาดถาดวัตถุดิบไม่เท่ากัน: ภาพถาดหมูและเต้าหู้มีสเกลรูปภาพใหญ่กว่าถาดอื่น ส่งผลให้กรอบ Hitbox คลิกไม่ครอบคลุม แก้ไขโดยการแยกอาร์เรย์พิกัดและขนาด (ingSizes) ประจำแต่ละถาดให้สอดคล้องกับขนาดภาพจริง

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

1. ภาพสไตล์พิกเซลอาร์ตและฟอนต์พิกเซล ให้บรรยากาศน่ารัก
2. นำระบบหลอดเวลาความอดทนของลูกค้าแบบถอยหลังมาผสานกับระบบหักลบยอดเงินจริง หากไม่ทันเวลาหรือเสิร์ฟผิดจะถูกหักเงินจนอาจเกิดภาวะล้มละลาย เพิ่มความท้าทายในการเล่น
3. สามารถรวมระบบ Game State Machine, การโหลดมัลติมีเดียแบบเธรดแยก, และการควบคุมตรรกะเกมที่ซับซ้อนไว้ในระบบ Java Swing ได้อย่างมีประสิทธิภาพ
4. มีเนื้อเรื่องเปิดเกมจากเจ้าของร้าน ไม่ใช่เริ่มเล่นทันที
5. สร้างปุ่มและ UI เองทั้งหมดด้วย Graphics2D

3.3 คำแนะนำและข้อเสนอแนะสำหรับการต่อยอด

อยากให้เวลาอาจารย์อธิบายโค้ดในห้อง ค่อยๆไปช้าๆค่ะ บางทีหนูเขียนตามแล้วไม่ค่อยทัน แต่ไม่ค่อยกล้ายกมือถาม เผื่อมีบางคนก็ไม่กล้าเหมือนกันค่ะ แต่ถ้าตั้งใจฟังอาจารย์อธิบายเข้าใจง่ายมากค่ะ